Running Application Task Plan

Background: As an avid runner and hiker, I could benefit from having data-driven insights into my fitness and running health to monitor my performance and guide next steps. My Garmin watch displays useful information including biometrics such as VO2 Max, BMI, Pulse Ox as well as race predictions anchored on predictive models, however, the reports are not consolidated into a single or easily navigable view in the web-based platform. In particular, there are a large number of menu icons in the left pane of the homepage from which metrics, indicators, and KPIs are broken down into multiple categories including “Activities”, “Health Stats”, “Performance Stats”, “Training and Planning”, “Insights”, and “Reports” requiring the user to have to navigate through multiple views to obtain a well-rounded picture of their performance and future predictions.

This is not streamlined, and thus, time-intensive making it difficult to paint a picture of the user’s overall health status. Furthermore, while the dashboards contain time-series forecasting trends for health data and race predictions, they lack true indicators and KPIs that are tied to performance objectives in a way that characterizes the athlete’s current status benchmarked against these objectives and healthy benchmarks (For example: A donut chart that displays a numerical health value for an athlete’s current VO2 Max or a model-driven score for their race prediction color-coded in reference to benchmarked domains (ex: Green for healthy, yellow for moderate, red for unhealthy). Many of these described features appear to be offered in the Garmin Connect + mode, however, that is a paid feature.

Objective: The purpose of the task is to build a running application with a dashboard that displays data-driven insights, forecasts, predictions, and other running metrics using historical data gathered from Garmin files. This dashboard will ideally display multiple features in a single view allowing for efficient navigation across different categories of models, indicators, and KPIs. In this way, the application is effectively a software product that replicates the end-to-end workflow of data ingestion, cleaning, modeling, and display of running activity data specifically.

Scope: The task will incorporate the use of Python for the development of the dashboard, visual features, any interfaces, as well as statistical models and metrics. Generative AI software including Cursor and Claude may be used to enhance productivity in areas such as requirements development, system design, code completion, debugging, unit and integration testing, version control, and others. The application will be developed for use in a MacOS environment and may later be deployed into AWS. The application will be compatible with Garmin Activity files of the “.gpx” format.

Requirements

The requirements are categorized into technical and functional using the following definitions:

1) **Technical Requirements** - provide guidance for developers to build the solution. They are typically precise and testable.
    a) Purpose: Define system behavior, constraints, and architecture
    b) Examples: Use specific programming language, API structure, database, comply with standard/regulation
    c) Characteristics: These are “shall” statements used in technical specifications (ex: The system shall process 1,000 transactions per second)

2) **Functional Requirements** - define the quality attributes and user perception of the system, which can be interpreted differently depending on who is evaluating them
    a) Purpose: Describe the user experience, performance, and reliability
    b) Examples: The system must allow users to login. The system must generate a monthly sales report. The user interface should be modern. The application must be easy to use. The system should be responsive.
    c) Characteristics: These are often harder to test directly without defining specific metrics

Within these categories, the requirements will be further categorized as “Must Have” and “Nice to Have”.

Technical Requirements:

1) Must Have

a) Python shall be the programming language used for dashboard, API, statistical analysis and modeling development
b) Cursor shall be used as the ADE (final)
c) Git and GitHub shall be used for version control (final)
d) The system shall only accept running activity file types of the .gpx format and fail with an error message if another activity type is input by the user
e) The system shall display an error message on the dashboard if an invalid numerical value is input for a range for a chart or model/metric/KPI calculation
f) The system shall automatically detect and handle missing or corrupt fields within a .gpx file, substituting null values and surfacing a warning rather than failing silently
g) The system shall store all parsed running activity data and model outputs locally in a structured format, avoiding repeated parsing of the same .gpx files on subsequent launches.

h) All tasks shall run as background processes so the UI remains responsive and interactive while models are being fitted or updated.
i) The dashboard shall allow for users to upload custom Garmin Activity files of ".gpx" filetype into a repository file path for analysis (final)
j) The system shall be evaluated using unit testing, integration testing, and user acceptance testing (verification and validation) from end-users (final)
k) The system shall derive computed fields such as pace, split times, elevation gain/loss, training load from raw .gpx data at ingestion time (final)
l) The system shall develop time-series models fit to user's historical data (final)

2) Nice to Have

a) The system shall generate race time predictions for standard distances (mile, 5K, 10 K, half-marathon, marathon) using a statistical model fitted to the user's historical pace and physiological data.
b) The system shall display a weekly and monthly training load metric visualised as a chart so the user can identify overtraining or undertraining periods
c) The system shall identify and flag anomalous runs, sessions where key metrics deviate significantly from the user's personal baseline, with a brief explanatory annotation on the chart.
d) The system shall project a personalised target race date for a user-specified goal time based on the current field trajectories and historical improvement rates.

Functional Requirements:

1) Must Have

a) The dashboard shall present all primary metric categories such as biometrics, race predictions, performance trends, and route maps, within a single scrollable view, avoiding navigation to separate pages.
b) Jira Kanban boards shall be used to track epics, tasks, bugs, and other project elements.
c) Versions and releases for the MacOSXX shall be documented in GitHub (final).
d) The dashboard shall be installable as a standalone macOS application (e.g. via a Python-bundled app or a local server launched by a shell script) without requiring the user to interact with a terminal after initial setup (final).

e) The dashboard shall allow the user to export visualizations, model metrics, indicators, and KPIs as an alternative to automatic data persistence.
f) The dashboard shall display some combination of the following metrics, indicators, and KPIs:

    a) **Metrics -** These are granular, objective measurements of a specific activity or physical state. They are the building blocks used to calculate your overall status.

- Activity Calories: Raw energy expenditure for a specific session.
- Average Heart Rate: The mean beats per minute during an activity.
- Average Pace: Speed expressed as time per unit of distance.
- Average Speed: Distance covered divided by time.
- Average GCT Balance: The symmetry of ground contact between left and right feet.
- Average Ground Contact Time: The duration your foot is on the ground.
- Average Run Cadence: Number of steps per minute.
- Average Stride Length: The physical distance between steps.
- Average Vertical Oscillation: The "bounce" in your running motion.
- Average Vertical Ratio: Vertical oscillation divided by stride length.
- Max Heart Rate: The highest recorded heart rate.
- Total Activity Time: The raw duration of a workout.
- Total Distance: The physical length of the activity.
- BMI: A simple calculation of weight relative to height.

    b) **Indicators -** These points take raw metrics and add context (like "High," "Low," or "Balanced") to tell you how you are performing or recovering relative to a baseline

- Calories Remaining: Contextualizes calories burned against a daily goal.
- HRV Status: Indicates your body's readiness/stress levels based on heart rate variability trends.
- Lactate Threshold: Indicates the intensity level where fatigue rapidly increases.

- Race Predictor: An indicator of potential future performance based on current data.
- Recovery Advisor: A retrospective indicator suggesting how long to rest after effort.
- Training Status: Summarizes whether your current load is "Productive," "Peaking," or "Strained."
- Workout Recommendation: An actionable suggestion based on recent training trends.

c) **KPIs -** These are high-level scores that define your overall "success" or fitness level. They are the primary goals athletes track to see if their long-term strategy is working.

- Endurance Score: A strategic measure of your long-term capacity for sustained effort.
- Fitness Age: A high-level interpretation of your overall health relative to your actual age.
- Hill Score: A specific KPI for climbing capability and strength.
- VO2 Max: The gold-standard KPI for cardiovascular fitness and aerobic power.

2) Nice to Have

a) All plots shall support interactive zoom and pan so the user can examine fine-grained intervals within a long run or compare multi-week trends.
b) The dashboard shall include a date-range selector that filters all data simultaneously, ensuring every metric displayed is consistent with the selected period.
c) The dashboard shall allow for users to upload custom Garmin Activity files of “.gpx” filetype through drag-and-drop feature in the UI
d) The dashboard shall expose a settings panel allowing the user to configure personal parameters such as resting HR, weight, goal race distance, that are used as inputs to the statistical models.

Milestones and Timelines

For a display of project milestones and timelines built into an Agile Kanban view, please navigate here.

Deliverables

The deliverables will be voiceover presentations at each major stage of the project (i.e. requirements analysis, system design, etc.) accompanied by descriptive text summarizing the development, important lessons learned, trade-offs and decisions, etc. They will be published on LinkedIn and youtube. Ultimately, GitHub releases intended for use on MacOS and later, a deployment in an AWS environment using Free Tier credits will be performed. The quantity, features, and timelines of the releases are evolving and will be updated over the course of the project.